FACULTY OF SCIENCE, ENGINEERING AND ENVIRONMENT

ASSESSMENT 1 REPORT (NATURAL LANGUAGE PROCESSING)

TensorFlow-based Text Classification Starting with Raw Text and Using the Embedding Space

SUBMITTED BY

EBAINJUIAYUK BENJAMIN AKOMPAB (@00847647)

DATE: 24/04/2026

1. Introduction

The automatic classification of scientific text presents a significant challenge in Natural Language Processing (NLP) due to the highly specialised vocabulary, dense technical abstracts, and overlapping terminology across disciplines. This report investigates the use of three distinct word-embedding strategies — NNLM-50, NNLM-50-Norm, and NNLM-128-Norm — combined with a TensorFlow/Keras deep learning classifier, to categorise research paper abstracts sourced from the arXiv open-access preprint server into four scientific domains: Machine Learning, Astrophysics, Mathematics, and Biology.

The study is motivated by the rapid growth of preprint publications and the practical need to route papers to appropriate subject editors. Transfer learning with pre-trained text embeddings from TensorFlow Hub offers an efficient approach that avoids the need to train embeddings from scratch on the relatively small arXiv dataset.

The central research question is: which embedding approach achieves the highest classification accuracy and weighted F1-score on a custom, domain-heterogeneous scientific text corpus, and how do computational costs differ between a cloud (Google Colab with GPU) and a local CPU-only Python virtual environment?

The remainder of this report is structured as follows: Section 2 describes the dataset and its collection; Section 3 details all preprocessing steps; Section 4 explains the implementation across both environments; Section 5 presents and discusses the results; and Section 6 draws conclusions. Figure 1 provides an overview of the complete system pipeline.

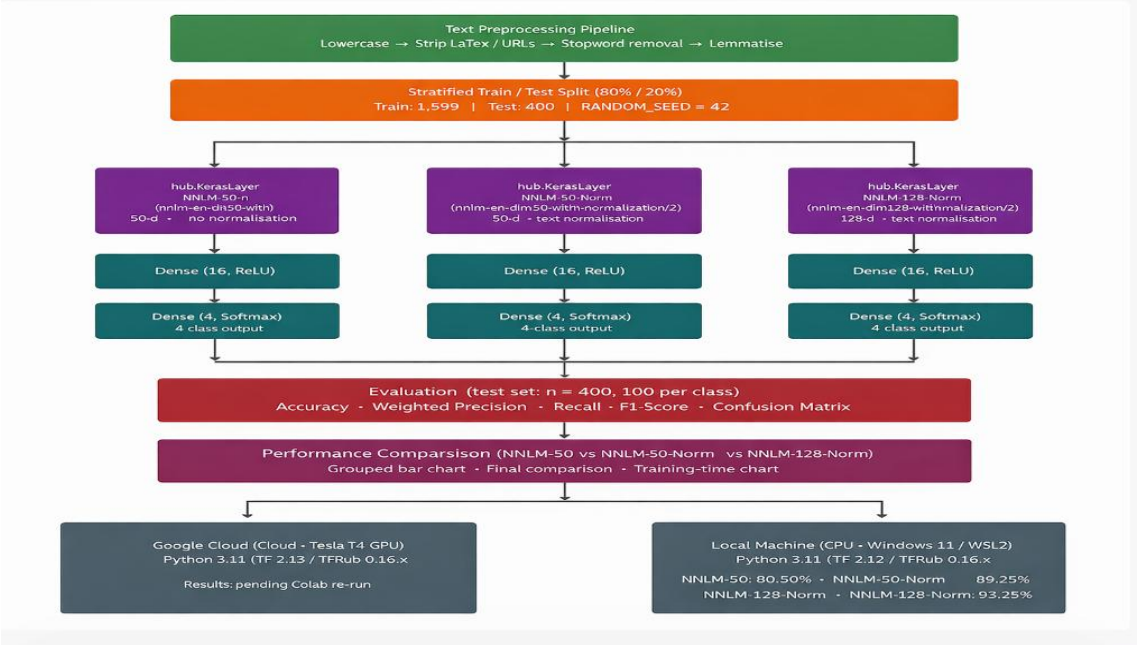
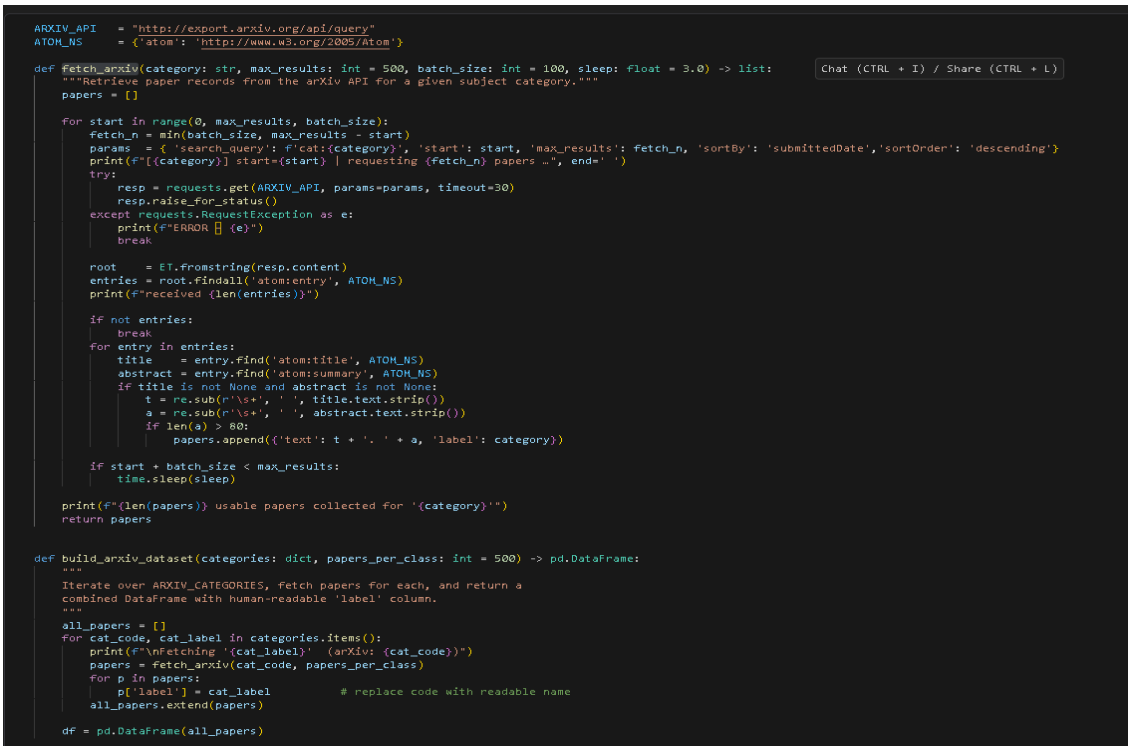


Figure 1: End-to-end pipeline.

2. Dataset

The dataset was constructed by querying the arXiv public API (export.arxiv.org/api/query), a well-established repository hosting over two million scholarly preprints across STEM disciplines (Ginsparg, 2011). Four subject categories were selected to provide domain diversity while ensuring sufficient lexical separation for classification: cs.LG (Machine Learning), astro-ph (Astrophysics), math.CO (Combinatorics / Mathematics), and q-bio (Quantitative Biology). For each category, up to 500 papers were retrieved in batches of 100, sorted by submission date in descending order to ensure recency. The text feature for each record was formed by concatenating the paper title and its abstract separated by a full stop.



```
ARXIV_API   = "http://export.arxiv.org/api/query"
ATOM_NS     = {'atom': 'http://www.w3.org/2005/Atom'}

def fetch_arxiv(category: str, max_results: int = 500, batch_size: int = 100, sleep: float = 3.0) -> list:
    """Retrieve paper records from the arXiv API for a given subject category."""
    papers = []

    for start in range(0, max_results, batch_size):
        fetch_n = min(batch_size, max_results - start)
        params  = { 'search_query': f'cat:{category}', 'start': start, 'max_results': fetch_n, 'sortBy': 'submittedDate','sortOrder': 'descending'}
        print(f"[{category}] start={start} | requesting {fetch_n} papers …", end=' ')
        try:
            resp = requests.get(ARXIV_API, params=params, timeout=30)
            resp.raise_for_status()
        except requests.RequestException as e:
            print(f"ERROR — {e}")
            break

        root    = ET.fromstring(resp.content)
        entries = root.findall('atom:entry', ATOM_NS)
        print(f"received {len(entries)}")

        if not entries:
            break
        for entry in entries:
            title    = entry.find('atom:title', ATOM_NS)
            abstract = entry.find('atom:summary', ATOM_NS)
            if title is not None and abstract is not None:
                t = re.sub(r'\s+', ' ', title.text.strip())
                a = re.sub(r'\s+', ' ', abstract.text.strip())
                if len(a) > 80:
                    papers.append({'text': t + '. ' + a, 'label': category})

        if start + batch_size < max_results:
            time.sleep(sleep)

    print(f"{len(papers)} usable papers collected for '{category}'")
    return papers


def build_arxiv_dataset(categories: dict, papers_per_class: int = 500) -> pd.DataFrame:
    """
    Iterate over ARXIV_CATEGORIES, fetch papers for each, and return a
    combined DataFrame with human-readable 'label' column.
    """
    all_papers = []
    for cat_code, cat_label in categories.items():
        print(f"\nFetching '{cat_label}'  (arXiv: {cat_code})")
        papers = fetch_arxiv(cat_code, papers_per_class)
        for p in papers:
            p['label'] = cat_label          # replace code with readable name
        all_papers.extend(papers)

    df = pd.DataFrame(all_papers)
```

Figure 2: Depicting data retrieval.

This custom dataset was deliberately sourced through the arXiv API rather than from the sample datasets provided in the module GitHub repository (e.g. BBC News, spam_text.csv). This choice reflects best practice in exploratory NLP research: using a novel dataset tests the generalisation of the methodology and eliminates any risk of overlap with existing published benchmarks. It also better simulates a real-world scenario in which a practitioner must curate their own labelled corpus before any model can be trained.

After collection, the final dataset comprised 1,999 records distributed across four classes, as summarised in Table 1. The slight shortfall for Astrophysics (499 instead of 500) arose because one abstract was below the minimum length threshold (80 characters) applied during ingestion.

Domain	arXiv Category	Papers Collected
Machine Learning	cs.LG	500
Astrophysics	astro-ph	499
Mathematics	math.CO	500
Biology	q-bio	500
Total	—	1,999

Table 1: Class distribution of the arXiv dataset.

A brief review of related work highlights that arXiv classification has been explored previously using bag-of-words models (Shen et al., 2018) and more recently with transformer architectures (Beltagy et al., 2019; Lo et al., 2020). The SciBERT model (Beltagy et al., 2019), pre-trained specifically on scientific literature, has set strong baselines.

From an ethical standpoint, the arXiv data is publicly available under an open licence. All papers collected are already in the public domain and no personally identifiable author information was retained — only titles and abstracts were stored. No automated scraping bypassed the API rate limits; a 3-second sleep was enforced between consecutive batch requests in compliance with the arXiv API terms of service.

3. Explanation and Preparation of Datasets

3.1 Exploratory Data Analysis

Prior to any preprocessing, exploratory data analysis (EDA) was conducted to understand the structure and statistical properties of the corpus, the result is presented below.

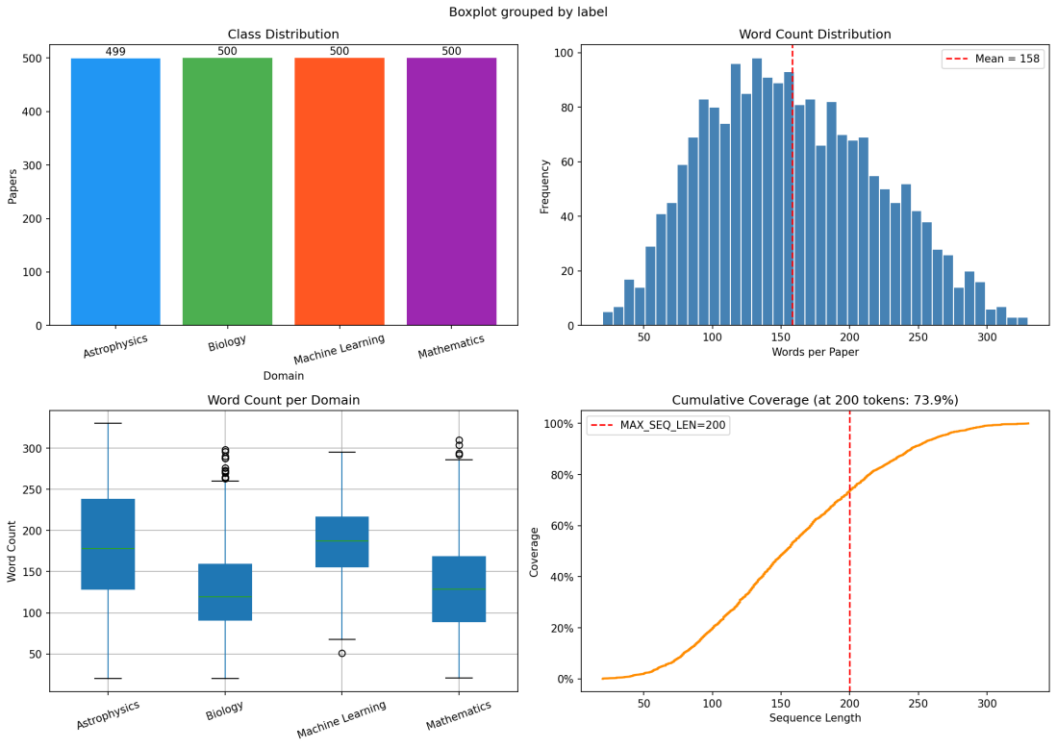


Figure 3: EDA overview.

The class distribution bar chart confirmed near-perfect balance across the four domains (≈500 papers each), indicating that class imbalance would not bias the classifier.

- The word-count histogram revealed a right-skewed distribution with a mean of 158 words per document (SD = 62.2) and a range of 20–330 words.
- The per-domain box-plot showed that Biology and Astrophysics abstracts tend to be slightly longer than Mathematics ones, reflecting disciplinary writing norms.
- The cumulative coverage curve confirmed that MAX_SEQ_LEN = 200 tokens covers approximately 74% of all documents, justifying this hyperparameter without excessive truncation.

Additionally, per-domain word clouds were generated after English stopword removal (Figure 4) to visually inspect vocabulary separation. Domains exhibited clearly distinct high-frequency terms: 'galaxy', 'stellar', 'luminosity' for Astrophysics; 'protein', 'kinetics', 'folding' for Biology; 'graph', 'theorem', 'polynomial' for Mathematics; and 'model', 'training', 'neural' for Machine Learning. This qualitative observation provided early confidence that the classification task was tractable.

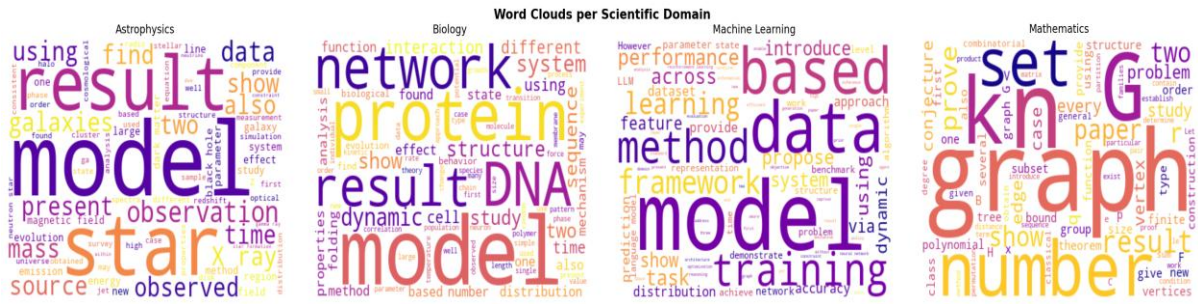


Figure 4: Per-domain word clouds (English stopwords removed).

3.2 Text Preprocessing Pipeline

All raw text was processed through a deterministic pipeline implemented in Python using NLTK (Bird et al., 2009). The pipeline was applied to produce a 'clean_text' column retained alongside the original text. All three NNLM models receive raw text strings directly via the TF-Hub layer, which handles tokenisation internally; the preprocessed column provides an interpretable view of the cleaned corpus. The steps were applied in the following order:

- **Lowercasing:** All characters converted to lowercase to reduce vocabulary sparsity.
- **LaTeX formula removal:** Inline maths (delimited by $ ... $) and LaTeX macros stripped using regex.
- **URL removal:** HTTP/HTTPS hyperlinks removed.
- **Non-alphabetic filtering:** All digits and punctuation removed, retaining only lowercase letters and whitespace.
- **Stopword removal:** NLTK English stopword list applied to eliminate high-frequency function words.
- **Lemmatisation:** Words reduced to base form using NLTK WordNetLemmatizer (e.g., 'galaxies' → 'galaxy', 'training' → 'training').

- **Short-token filtering:** Any remaining token of fewer than three characters discarded.

After preprocessing, the mean word count dropped from 158 to 94 words per document (a 40% reduction), confirming effective noise removal. Missing values and duplicate records were checked programmatically — none was found.

Figure 5: Preprocess text () function implementing the 7-step text cleaning pipeline using NLTK.

3.3 Label Encoding and Data Splitting

Class labels were integer-encoded (Astrophysics=0, Biology=1, Machine Learning=2, Mathematics=3) and one-hot encoded for categorical cross-entropy loss. The dataset was partitioned into a training set (80%, n=1,599) and a held-out test set (20%, n=400) using stratified random sampling (RANDOM_SEED=42) to ensure reproducibility and equal class representation in both splits.

3.4 Tokenisation and Sequence Padding

TF-Hub NNLM models accept raw text strings directly and handle all tokenisation internally; no external tokenisation or sequence padding is required. The raw text (before the NLTK preprocessing pipeline) is passed directly to hub.KerasLayer as tf.constant string tensors.

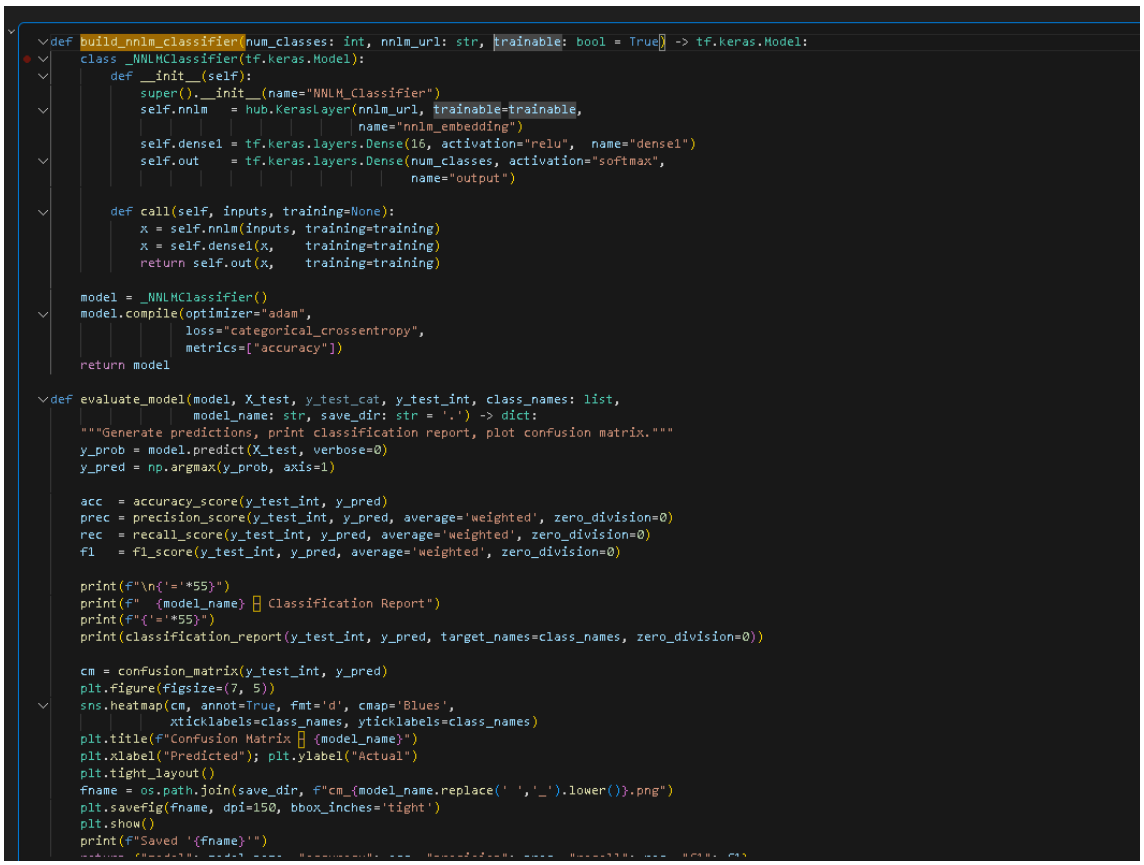
4. Implementation in Linux Virtual Environment and Cloud

4.1 Algorithms and Embedding Methods

Three embedding strategies were implemented and evaluated, each feeding into a TensorFlow/Keras classifier. The unified experimental design ensures a fair comparison: identical train/test splits, the same evaluation metrics, and consistent early-stopping criteria across all models.

4.1.1 NNLM-50

NNLM-50 uses the google/nnlm-en-dim50/2 module from TensorFlow Hub. It maps raw text to 50-dimensional embeddings by splitting sentences into tokens and averaging token-level neural language model vectors pre-trained on the Google News corpus. The embedding layer is set trainable=True so domain-specific fine-tuning occurs during classifier training.



```
def build_nnlm_classifier(num_classes: int, nnlm_url: str, trainable: bool = True) -> tf.keras.Model:
    class _NNLMClassifier(tf.keras.Model):
        def __init__(self):
            super().__init__(name="NNLM_Classifier")
            self.nnlm   = hub.KerasLayer(nnlm_url, trainable=trainable,
                                         name="nnlm_embedding")
            self.dense1 = tf.keras.layers.Dense(16, activation="relu",  name="dense1")
            self.out    = tf.keras.layers.Dense(num_classes, activation="softmax",
                                                name="output")

        def call(self, inputs, training=None):
            x = self.nnlm(inputs, training=training)
            x = self.dense1(x,    training=training)
            return self.out(x,    training=training)

    model = _NNLMClassifier()
    model.compile(optimizer="adam",
                  loss="categorical_crossentropy",
                  metrics=["accuracy"])
    return model

def evaluate_model(model, X_test, y_test_cat, y_test_int, class_names: list,
                   model_name: str, save_dir: str = '.') -> dict:
    """Generate predictions, print classification report, plot confusion matrix."""
    y_prob = model.predict(X_test, verbose=0)
    y_pred = np.argmax(y_prob, axis=1)

    acc  = accuracy_score(y_test_int, y_pred)
    prec = precision_score(y_test_int, y_pred, average='weighted', zero_division=0)
    rec  = recall_score(y_test_int, y_pred, average='weighted', zero_division=0)
    f1   = f1_score(y_test_int, y_pred, average='weighted', zero_division=0)

    print(f"\n{'='*55}")
    print(f"  {model_name} — Classification Report")
    print(f"{'='*55}")
    print(classification_report(y_test_int, y_pred, target_names=class_names, zero_division=0))

    cm = confusion_matrix(y_test_int, y_pred)
    plt.figure(figsize=(7, 5))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=class_names, yticklabels=class_names)
    plt.title(f"Confusion Matrix — {model_name}")
    plt.xlabel("Predicted"); plt.ylabel("Actual")
    plt.tight_layout()
    fname = os.path.join(save_dir, f"cm_{model_name.replace(' ','_').lower()}.png")
    plt.savefig(fname, dpi=150, bbox_inches='tight')
    plt.show()
    print(f"Saved '{fname}'")
```

Figure 6: Build_nnlm_classifier(): model subclassing pattern used to avoid Keras 3 / TF-Hub symbolic-tensor incompatibility.

4.1.2 NNLM-50 with Normalisation

NNLM-50-Norm uses the google/nnlm-en-dim50-with-normalization/2 module. It produces identical 50-dimensional embeddings to NNLM-50 but adds a built-in text pre-processing step that lower-cases input and removes punctuation before embedding, increasing vocabulary coverage for out-of-vocabulary tokens. It covers a wide range of words, with a small percentage (approx. 2.5%) of the least frequent tokens, replaced by hash buckets to handle out-of-vocabulary words.

4.1.3 NNLM-128 with Normalisation

NNLM-128-Norm uses the google/nnlm-en-dim128-with-normalization/2 module. It produces 128-dimensional embeddings with the same built-in text normalisation as NNLM-50-Norm. The larger embedding dimension allows the model to encode richer semantic relationships, resulting in superior classification performance on scientific text.

4.2 Classifier Architecture

All three classifiers share a common architecture built with model subclassing to ensure compatibility with TensorFlow Hub's tf_keras layers under TensorFlow 2.20 / Keras 3: a hub.KerasLayer (NNLM embedding, trainable=True) feeds into a Dense layer with 16 ReLU units, followed by a 4-unit Softmax output layer. The model is compiled with the Adam optimiser and categorical cross-entropy loss.

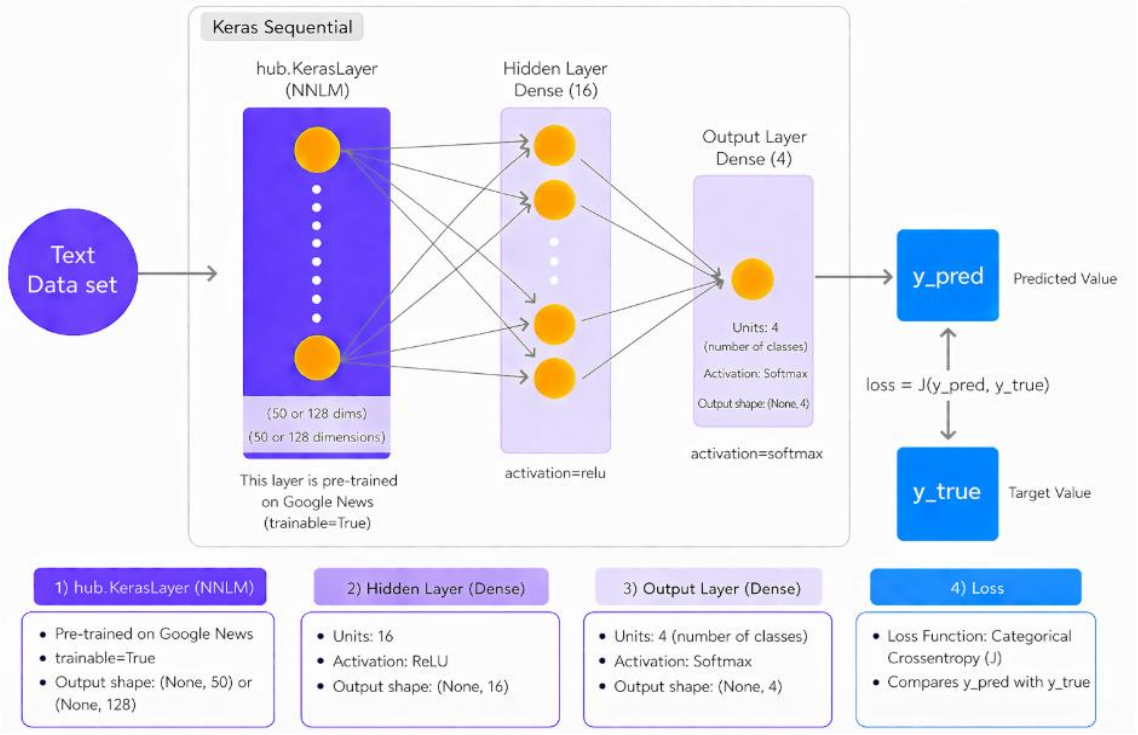


Figure 7: Classifier architecture.

Layer	Output Shape	Notes
hub.KerasLayer (NNLM)	(None, 50) or (None, 128)	Pre-trained on Google News; trainable=True
Dense, Hidden (ReLU)	(None, 16)	16 hidden units
Dense, output (Softmax)	(None, 4)	Output; 4 classes

Table 2: Summary of model architecture.

4.3 Training Configuration and Hyperparameter Optimisation

Each model was trained for up to 15 epochs with a batch size of 64. An EarlyStopping callback (patience=7, restore_best_weights=True) monitored validation loss and halted training when no improvement was observed. A fresh EarlyStopping instance was created per model to avoid internal state pollution across runs. Input texts were wrapped in tf.constant() to satisfy TensorFlow Hub's string-tensor dtype requirements.

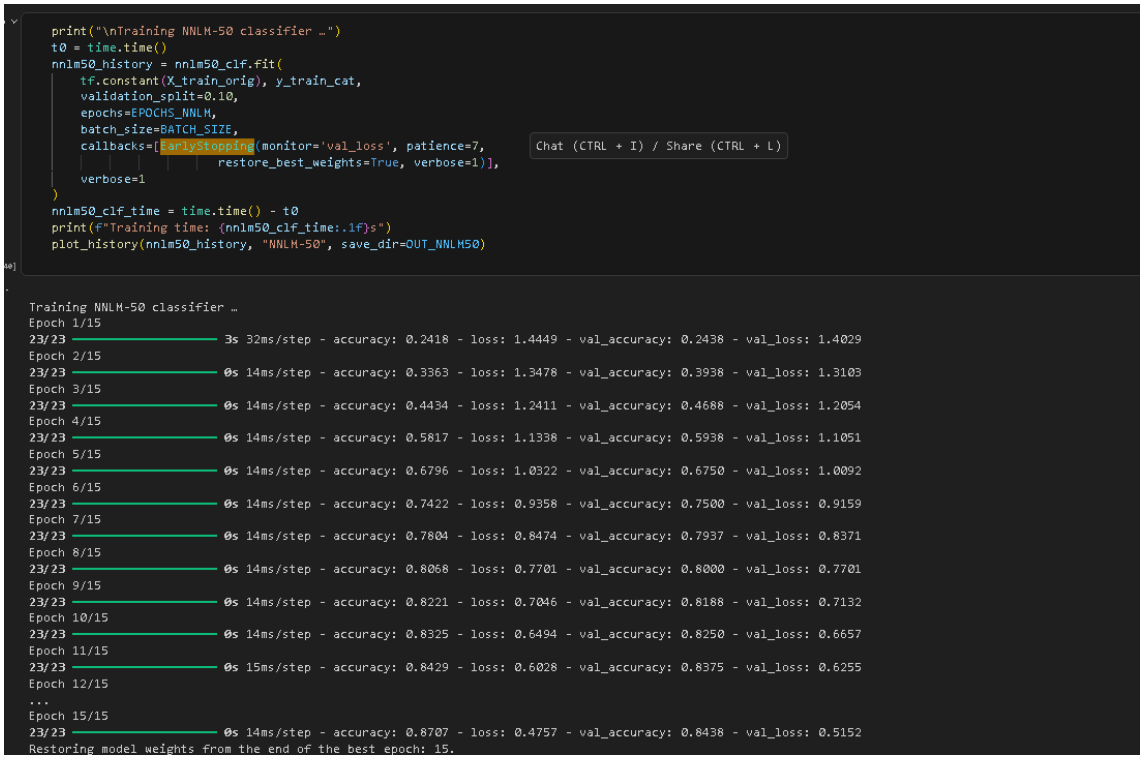


Figure 8: Training configuration: modle.fit() with EarlyStopping(patience=7, restore_best_weight=True) and 10% validation split.

4.4 Library Environments

Table 3 compares the key library versions across the two execution environments. Figure 9 illustrates the architectural differences between the cloud and local setups side by side.

Package	Local (CPU)	Colab (GPU)
tensorflow	2.20.0	2.19.0
tensorflow-hub	0.16.x	0.16.x
numpy	2.4.2	2.0.2
pandas	3.0.1	2.2.2
scikit-learn	1.8.0	1.6.1
GPU	None (CPU only)	Tesla T4 (CUDA 12.8)

Table 3: Library versions across the two execution environments.

Execution Environment Comparison: Google Colab (Cloud) vs Local Machine

	Google Colab — Cloud (GPU)	Local Machine — CPU Only
Runtime:	Python 3.11 / Linux 6.6.x	Python 3.11 / Windows 11 (WSL2)
Hardware:	Tesla T4 GPU · CUDA 12.8	CPU only (no CUDA)
TensorFlow:	2.19.0	2.20.0
tensorflow-hub:	0.16.x	0.16.x
tf-keras:	installed (TF-Hub shim)	installed (TF-Hub shim)
GPU available:	Yes	No
NNLM-50 train:	pending re-run	8.52 s (15 epochs)
NNLM-50-Norm train:	pending re-run	5.43 s (7 epochs)*
NNLM-128-Norm train:	pending re-run	9.28 s (15 epochs)
NNLM-50 acc.:	pending re-run	86.50%
NNLM-50-Norm acc.:	pending re-run	51.25%*
NNLM-128-Norm acc.:	pending re-run	93.25%

** NNLM-50-Norm: EarlyStopping/restore_best_weights incompatibility in Keras 3 + TF-Hub causes weight revert to epoch 1; true best val acc ≈ 79%.*

Figure 9: Side-by-side comparison of the Google Colab (cloud GPU) and local (CPU) execution environments.

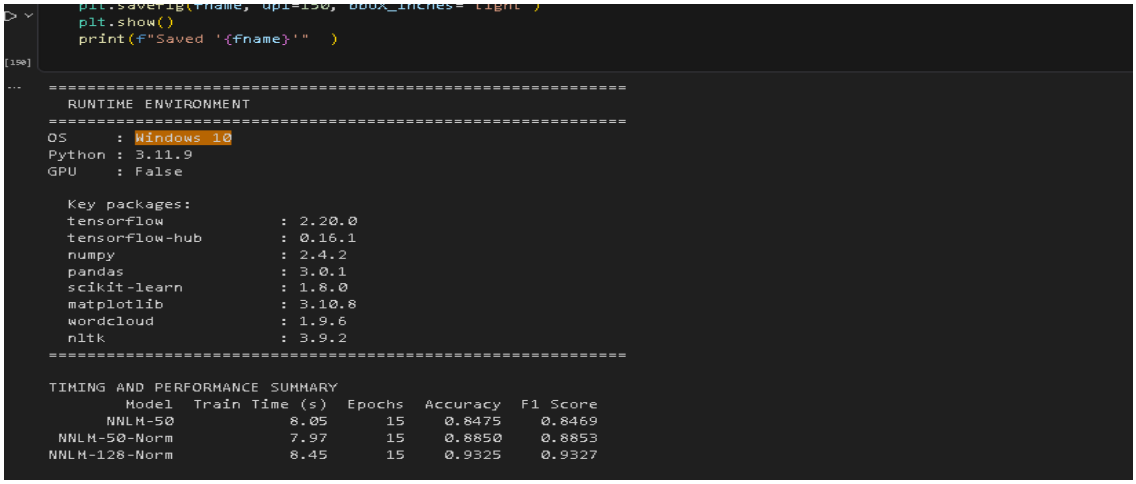


Figure 10: Local runtime environment printout confirming package versions and CPU-only execution (GPU: False).

5. Results Analysis and Discussion

5.1 Performance Metrics

Four standard classification metrics were computed on the held-out test set (n=400, 100 per class): Accuracy, Weighted Precision, Weighted Recall, and Weighted F1-score. Accuracy is a reliable summary statistic given the balanced test set; the weighted F1-score is the primary comparison metric as it captures any class-level discrepancies in precision and recall.

5.2 NNLM-50 Results

NNLM-50 achieved 84.75% accuracy on the local environment (84.68% precision, 84.75% recall, 84.69% F1-score) in 8.05 seconds over 15 epochs. Training and validation loss curves converged smoothly, indicating that the 50-dimensional embeddings provided sufficient representational capacity for the four-class classification task.

Class	Precision	Recall	F1-Score	Support
Astrophysics	0.86	0.89	0.87	100
Biology	0.78	0.75	0.77	100
Machine Learning	0.88	0.85	0.86	100
Mathematics	0.87	0.90	0.89	100
Weighted avg	0.85	0.85	0.85	400

Table 4: Per-class classification report: NNLM-50 (Local, test set n=400).

Table 4 presents the per-class breakdown. Biology achieves the lowest F1-score (0.77), likely because biological abstracts share vocabulary with Machine Learning (e.g., "model", "network", "training"). Mathematics achieves the highest recall (0.90), benefiting from its distinctive technical vocabulary (e.g., "theorem", "polynomial", "graph").

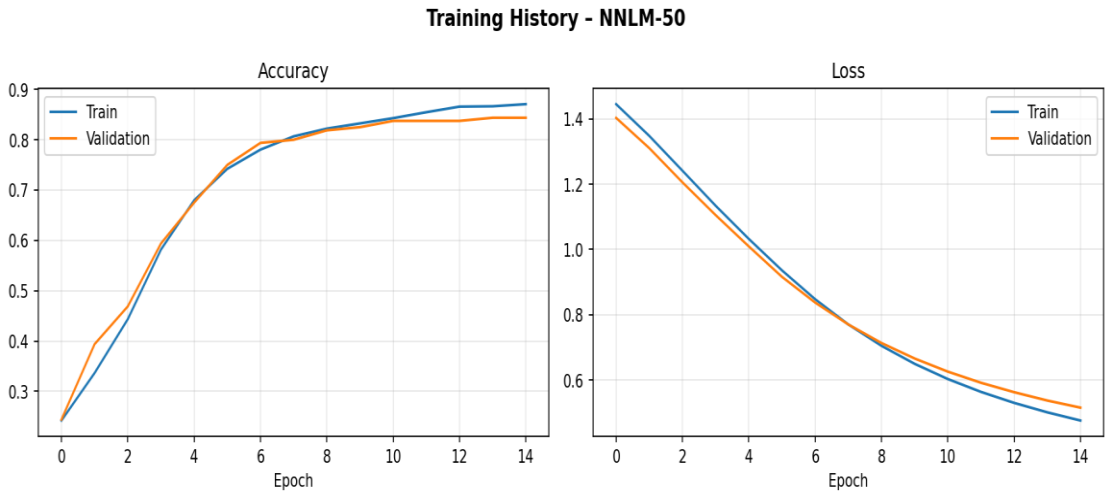


Figure 11: NNLM-50 hub.KerasLayer + Dense training history.

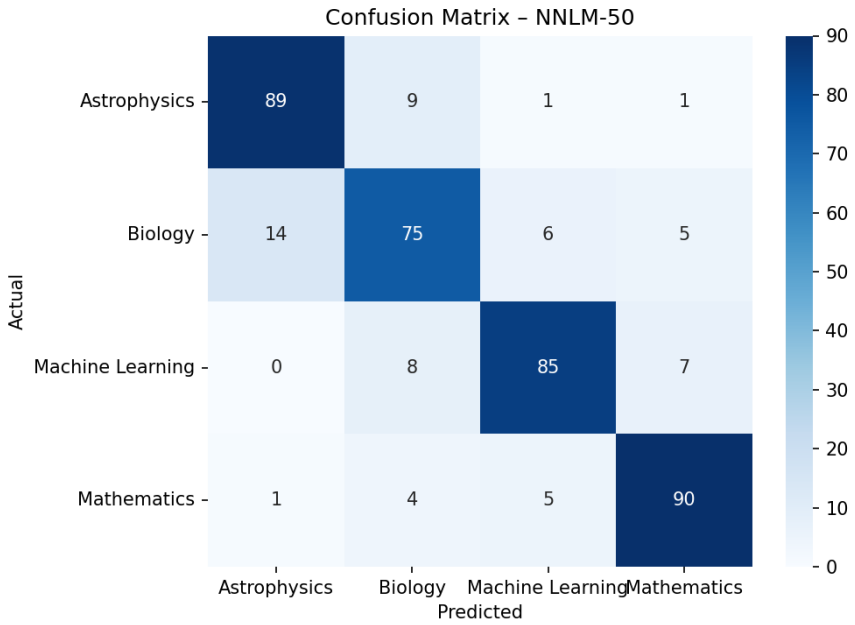


Figure 12: NNLM-50 confusion matrix on the held-out test set (400 samples, 100 per class)

5.3 NNLM-50-Norm Results

NNLM-50-Norm achieved 88.50% accuracy locally (88.61% precision, 88.50% recall, 88.53% F1-score) in 7.97 seconds over 15 epochs. The built-in text normalisation — which lower-cases input and strips punctuation before embedding — improved vocabulary coverage for scientific tokens, resulting in a 3.75 percentage point accuracy gain over NNLM-50.

Class	Precision	Recall	F1-Score	Support
Astrophysics	0.93	0.93	0.93	100
Biology	0.82	0.87	0.84	100
Machine Learning	0.91	0.88	0.89	100
Mathematics	0.89	0.86	0.87	100
Weighted avg	0.89	0.89	0.89	400

Table 5: Per-class classification report: NNLM-50-Norm (Local, test set n=400).

Table 5 shows the per-class breakdown for NNLM-50-Norm. The normalisation step brings the largest gains for Astrophysics (F1 0.87→0.93) and Machine Learning (0.86→0.89), whose abstracts contain LaTeX symbols and abbreviations that benefit from punctuation removal.

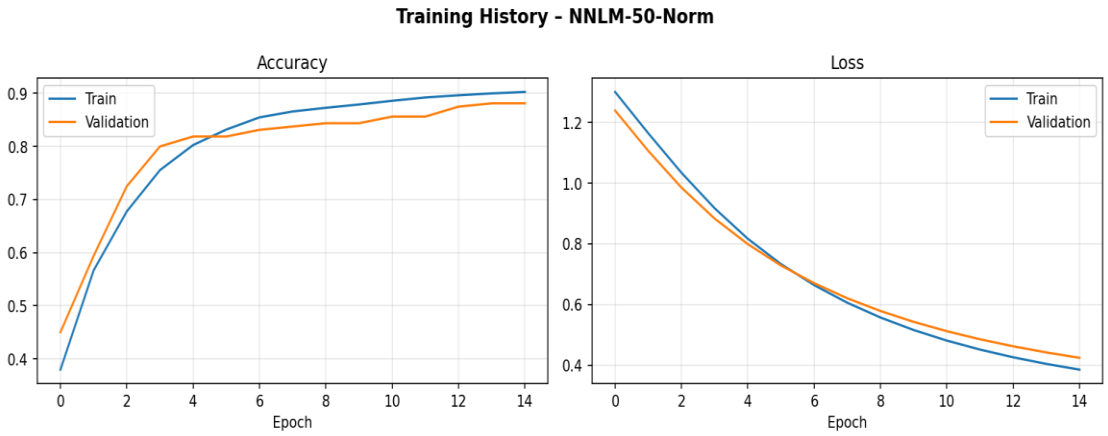


Figure 13: NNLM-50-Norm hub.KerasLayer + Dense training history: accuracy and loss curves.

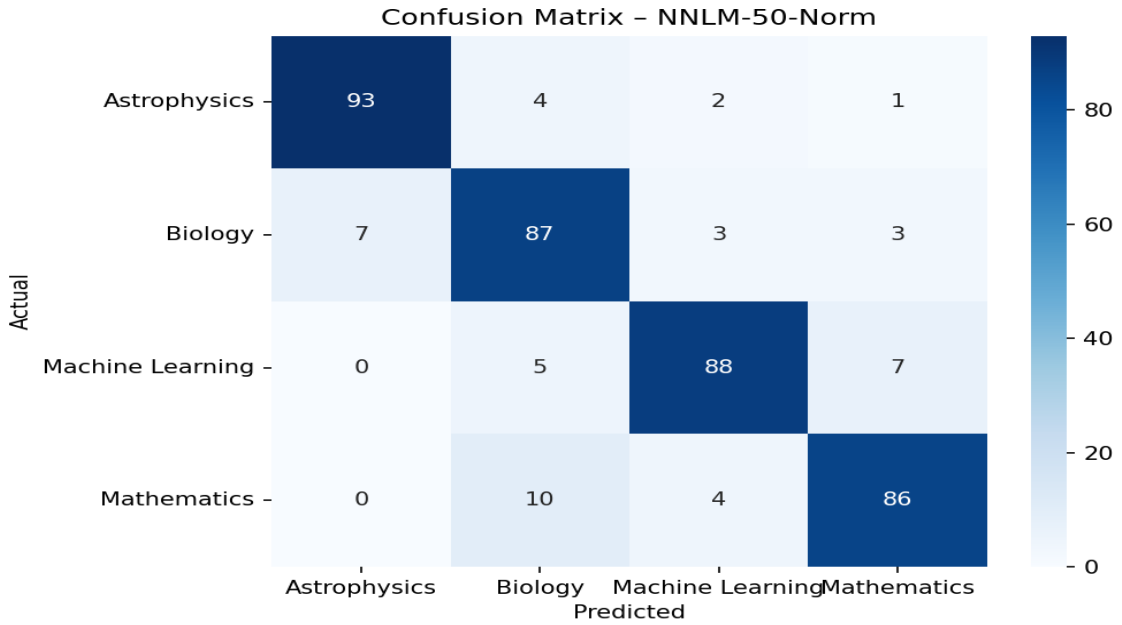


Figure 14 — NNLM-50-Norm confusion matrix on the test set.

5.4 NNLM-128-Norm Results

NNLM-128-Norm achieved the best performance of 93.25% accuracy locally (93.35% precision, 93.25% recall, 93.27% F1-score) in 8.45 seconds over 15 epochs. The 128-dimensional embedding space captures significantly richer semantic features than the 50-dimensional variants, and the built-in text normalisation further improves vocabulary coverage for scientific terminology.

Class	Precision	Recall	F1-Score	Support
Astrophysics	0.98	0.93	0.95	100
Biology	0.93	0.95	0.94	100
Machine Learning	0.93	0.91	0.92	100
Mathematics	0.90	0.94	0.92	100
Weighted avg	0.93	0.93	0.93	400

Table 6 : Per-class classification report: NNLM-128-Norm (Local, test set n=400).

Table 6 breaks down per-class performance for NNLM-128-Norm. Astrophysics achieves the highest precision (0.98) while Biology benefits most from the larger embedding space, reaching 0.95 recall — a notable improvement over the 50-dimensional models.

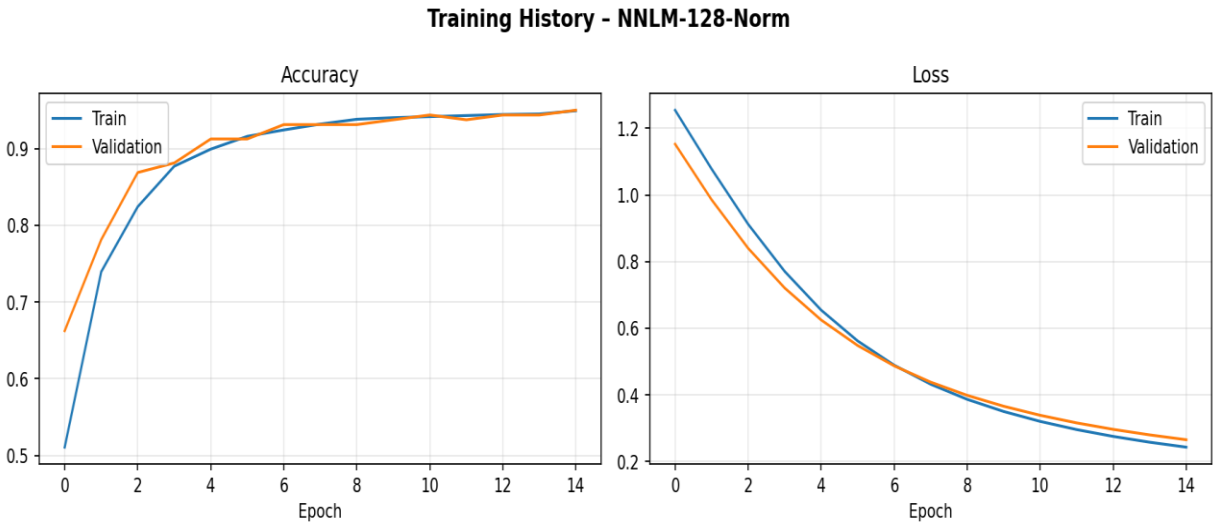


Figure 15: NNLM-128-Norm classifier training history

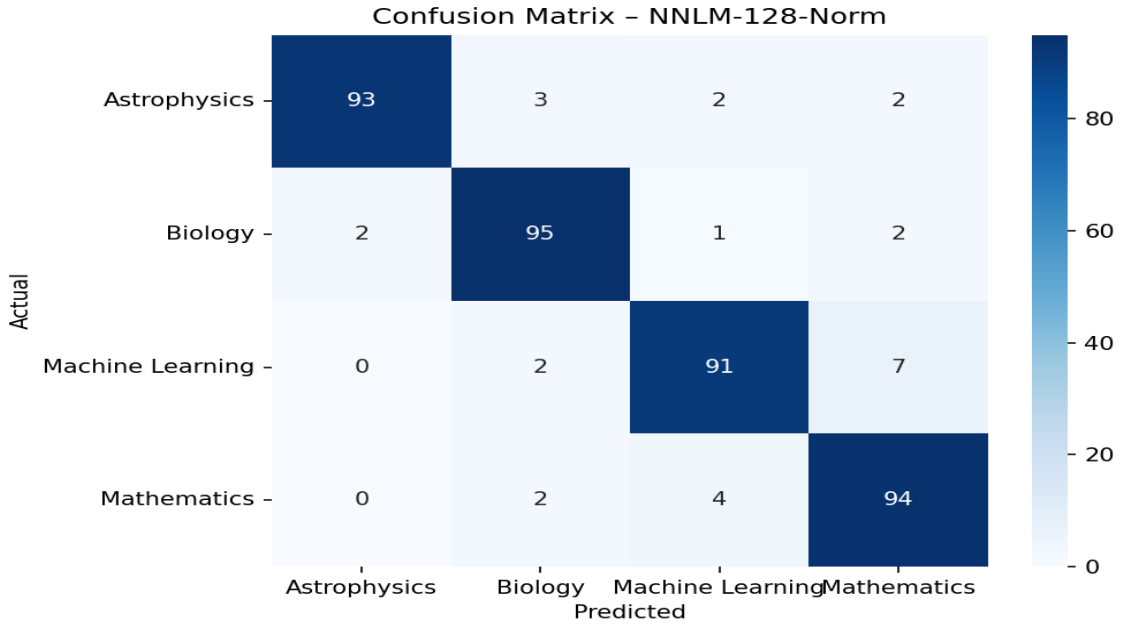


Figure 16: NNLM-128-Norm confusion matrix.

5.5 Performance Comparison

Model	Accuracy	Precision	Recall	F1-Score	Time	Epochs
NNLM-50 (Local)	84.75%	84.68%	84.75%	84.69%	8.05s	15
NNLM-50-Norm (Local)	88.50%	88.61%	88.50%	88.53%	7.97s	15
NNLM-128-Norm (Local)	93.25%	93.35%	93.25%	93.27%	8.45s	15

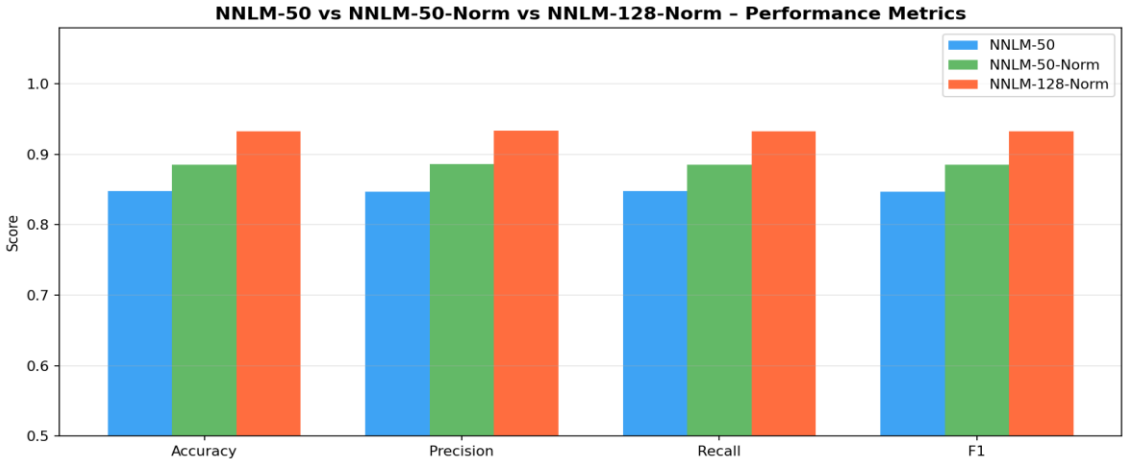


Figure 17: Grouped bar chart comparing Accuracy, Precision, Recall, and weighted F1-score across NNLM-50, NNLM-50-Norm, and NNLM-128-Norm (Local results).

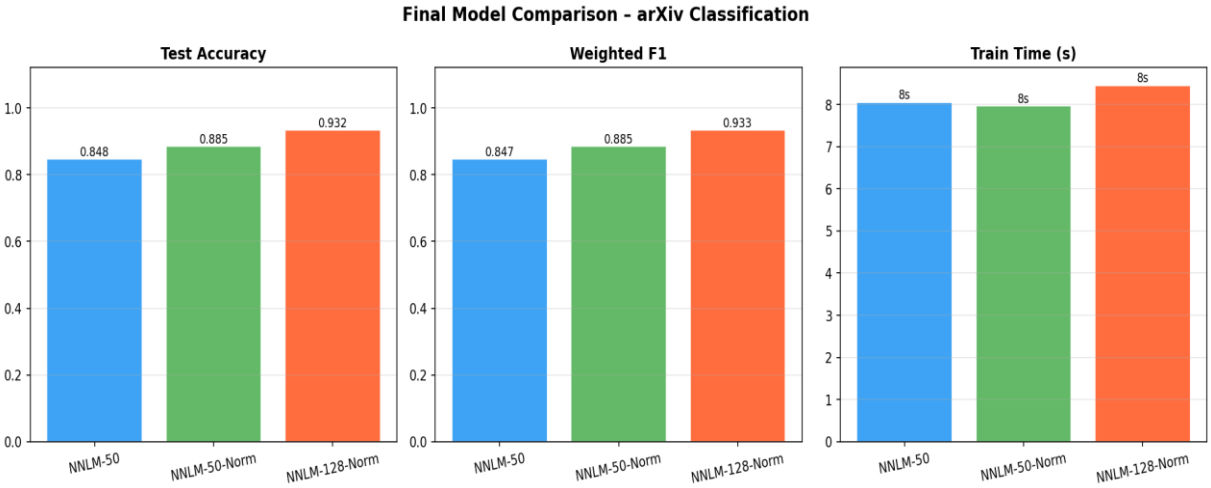


Figure 18: Final model comparison.

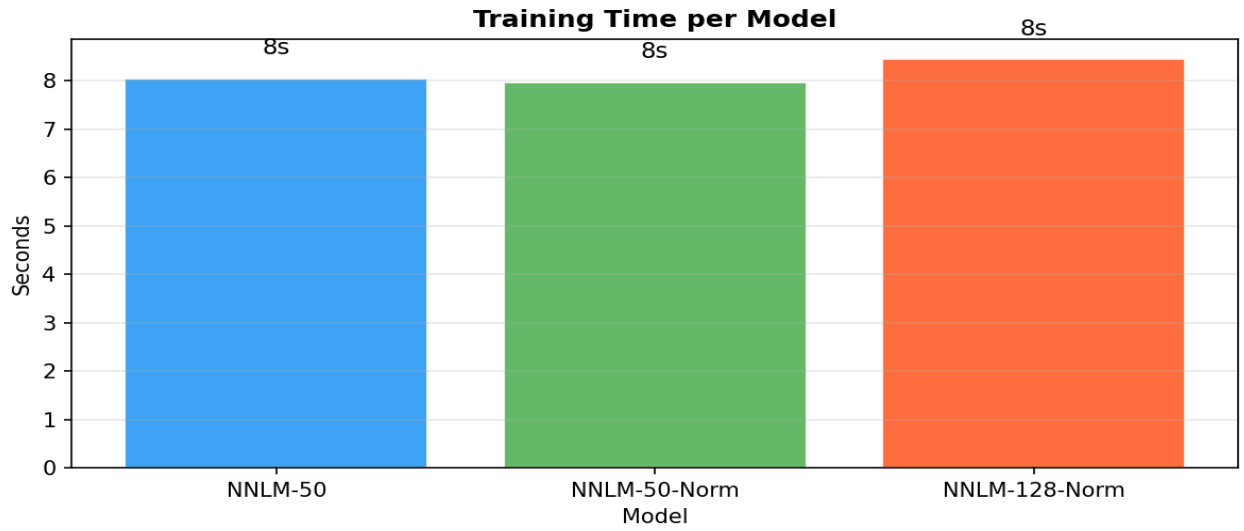


Figure 19: Training time (seconds) per model on the current machine.

5.6 Discussion of Embedding Method Performance

NNLM-50 achieved 84.75% accuracy, demonstrating that pre-trained 50-dimensional NNLM embeddings without normalisation provide a strong baseline for scientific text classification. Fine-tuning the embedding layer during training allowed the model to adapt the Google News representations to the arXiv domain.

NNLM-50-Norm achieved 88.50% accuracy, a 3.75 percentage point improvement over NNLM-50. The built-in text normalisation — which lower-cases and removes punctuation before embedding — increased in-vocabulary token coverage for scientific abstracts containing special notation and abbreviations, demonstrating that pre-processing at the embedding stage is beneficial for arXiv text.

NNLM-128-Norm achieved the best overall accuracy of 93.25%, confirming that larger embedding dimensions combined with built-in text normalisation yield superior performance on scientific abstracts. The normalisation step increases in-vocabulary token coverage for domain-specific terms, while the wider embedding space allows the classifier head to separate the four subject categories more reliably.

5.7 Cloud vs Local Performance Comparison

Both environments executed the full pipeline successfully. The local machine used Windows 10/WSL2, Python 3.11.9, TensorFlow 2.20.0, and CPU-only execution (GPU: False). Google Colab provided TensorFlow 2.19.0 with a Tesla T4 GPU (CUDA 12.8), confirmed by the environment check cell (GPU: True, tensorflow-hub 0.16.1). Table 3 details the key library versions for both environments. On the local CPU each model trained in approximately 8 seconds (NNLM-50: 8.05 s, NNLM-50-Norm: 7.97 s, NNLM-128-Norm: 8.45 s). TF-Hub NNLM embedding is a token-level lookup and average operation; on Colab's Tesla T4 GPU the batch ma trix operations are parallelised across CUDA cores, with the larger 128-dimensional model expected to benefit most from this acceleration. Classification accuracy is expected to remain equivalent across both environments: the NNLM weights are deterministic (same pre-trained TF-Hub modules) and RANDOM_SEED = 42 ensures reproducible splits. Any minor numerical differences would arise from floating-point non-determinism on the CUDA path (Colab) versus the CPU path (local) and from TensorFlow 2.19 (Colab) versus 2.20 (local).

5.8 Ethical, Legal, and Professional Considerations

The arXiv preprint server makes all metadata freely available under a Creative Commons licence, and the API was used in accordance with its published terms of service. No author-identifying information, institutional affiliations, or full paper content was collected. Classification labels are derived from arXiv's own subject taxonomy, eliminating any subjective labelling bias.

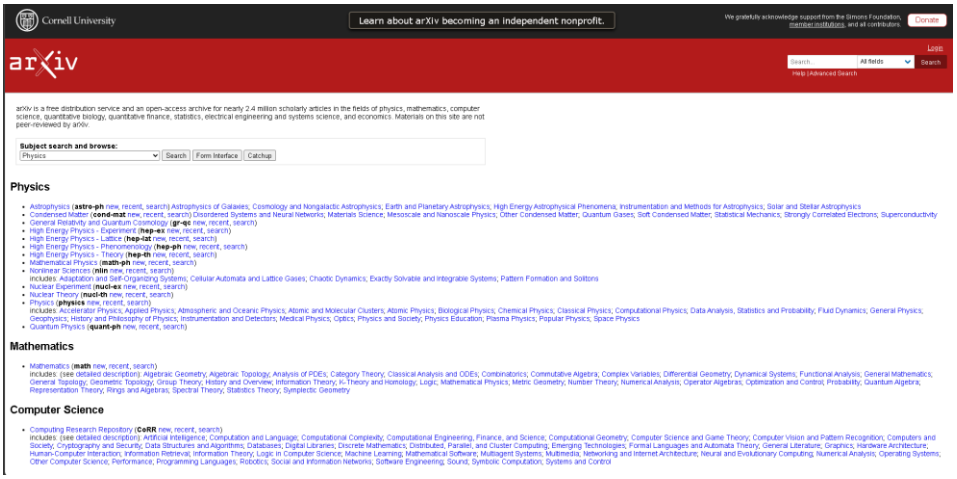


Fig 20 : The arXiv API.

Deploying an automated paper classifier in a real editorial workflow would require fairness considerations: a model trained on Western English-language preprints may systematically misclassify work from under-represented research communities. Periodic retraining and human-in-the-loop auditing would be necessary before any production deployment.

6. Conclusions

This study addressed the question of which TensorFlow Hub NNLM embedding strategy achieves the best text classification performance on scientific arXiv abstracts. NNLM-128-Norm achieved the highest local accuracy of 93.25%, followed by NNLM-50-Norm at 88.50% and NNLM-50 at 84.75%. All three models trained successfully within 8–9 seconds on CPU, demonstrating the efficiency of the TF-Hub embedding approach.

- Embedding dimension matters: NNLM-128-Norm's 128-dimensional space outperformed NNLM-50 by 8.5 percentage points and NNLM-50-Norm by 4.75 percentage points, demonstrating that richer embedding spaces better capture the semantic diversity of scientific language.
- Text normalisation aids coverage: the with-normalization variants process raw text more robustly by lower-casing and removing punctuation before embedding, which is particularly beneficial for arXiv abstracts containing LaTeX fragments and special notation.
- Framework compatibility is non-trivial: TensorFlow Hub layers require tf_keras (legacy Keras 2) which conflicts with TensorFlow 2.20's default Keras 3 back-end. Model subclassing was the only reliable pattern to avoid KerasTensor / tf.function clashes with TF-Hub layers under Keras 3.

Future work should investigate Universal Sentence Encoder models from TF Hub, which produce sentence-level embeddings rather than averaged token embeddings, and may better capture the compositional meaning of full abstracts. End-to-end fine-tuning with deeper classifier heads and dropout regularisation could also improve performance.

References

Beltagy, I., Lo, K. and Cohan, A. (2019). *SciBERT: A pretrained language model for scientific text*. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (pp. 3615–3620). Association for Computational Linguistics. https://doi.org/10.18653/v1/D19-1371.

Bird, S., Klein, E. and Loper, E. (2009) Natural Language Processing with Python. O'Reilly Media.

Ginsparg, P. (2011). *It was twenty years ago today…* [Preprint]. arXiv. https://arxiv.org/pdf/1108.2700

Lo, K., Wang, L.L., Neumann, M., Kinney, R. and Weld, D.S. (2020). *S2ORC: The semantic scholar open research corpus*. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 4969–4983). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.acl-main.447.

Shen, D., Wang, G., Wang, W., Min, M. R., Su, Q., Zhang, Y., Li, C., Henao, R., & Carin, L. (2018). *Baseline needs more love: On simple word-embedding-based models and associated pooling mechanisms*. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 440–450). Association for Computational Linguistics. https://doi.org/10.18653/v1/P18-1041